RX LFSR Functional Design Specifications

UCIe 3.0 physical layer

Version 1

Block Owner

ENG. Karim Waseem

Authors

Hussien Mohamed Hussien

This Page is left Blank Intentionally

1 Table of Contents

2 Revision History

Version	Date	Author(s)	Revision Notes	Owner Approval

3 Overview

This block is responsible for descrambling incoming data and also in training responsible for pattern detection

4 Operation and Description

4.1 Digital Interface

4.1.1 Parameters Names

Parameter Name	Default	Description
pDATA_WIDTH	64	It defines the width of the data input & output ports
pLANE_ID_SEED	24	It says the seed specified per lane

4.1.2 Ports Names

Port Name	Port Width	Port Type	Description
i_data_in	pDATA_WIDTH	input	Data input
i_error_threshold	16	input	It says the maximum error count that can occur
i_clk	1	Input	The clock used in the design
i_enable	1	Input	Enable for the block
i_reset	1	input	Reset of the system
i_train	1	input	Signal to identify if the data is destined to be descrambled or be compared
o_lane_success	1	output	Signal is asserted if the pattern is successful
o_data_out	pDATA_WIDTH	output	Output data

4.2 Functional Description

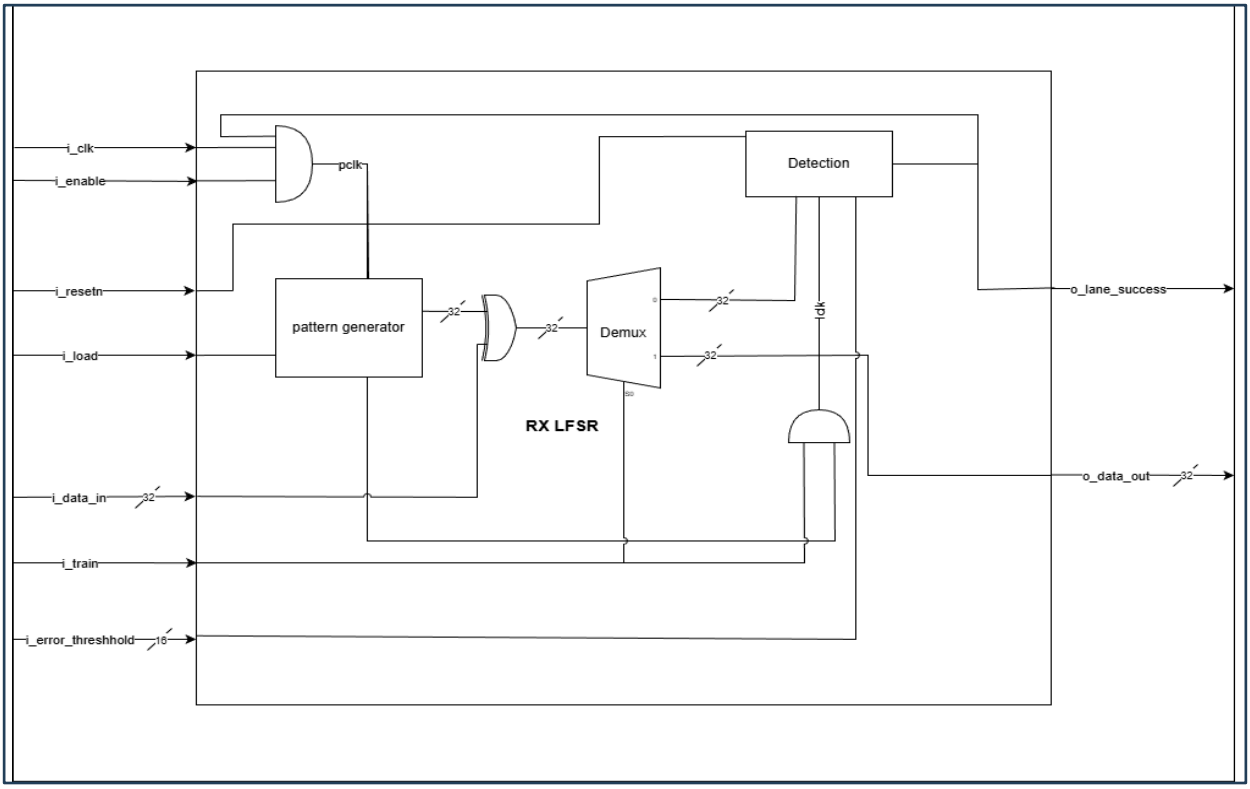
- The block diagram is responsible for the descrambling data or detecting pattern generated according to signal i_train
- The core of the pattern generator is a **23-bit maximal-length LFSR**. The feedback function cal() advances the register by one step, implementing the recurrence defined by the tap positions at bits **22, 20, 15, 7, 4, and 1** of the current state. This corresponds to the primitive polynomial:

  $$x^{23} + x^{21} + x^{16} + x^{8} + x^{5} + x^{2} + 1$$

  producing a sequence of length $2^{23} - 1$ before repeating.
- Rather than outputting one bit per clock cycle, the module unrolls pDATA_WIDTH successive LFSR iterations **combinationally** in a single cycle:
  1. next[0] is set to the current LFSR state.
  2. Each next[i+1] is computed by applying cal() to next[i].
  3. The **MSB (bit 22)** of each intermediate state is extracted and assigned to pattern[i].
  4. The result is that pattern presents pDATA_WIDTH consecutive LFSR output bits simultaneously on every clock edge, equivalent to what a serial LFSR would produce over pDATA_WIDTH individual cycles.

- On every rising edge of pclk:
    - If i_load is asserted → the LFSR is **synchronously reloaded** with pLANE_ID_SEED, restarting the sequence from a known state.
    - Otherwise → the LFSR advances by **pDATA_WIDTH steps** in one cycle, storing next[pDATA_WIDTH] as the new state, consistent with the parallel output just produced.
- If we are in training mode, the data will enter detection block which will detect if the data has error by xoring with the local pattern if there is mismatch the counter of errors will increment then will compare the total count of errors with the i_error_threshold and if it is greater than the threshold the block will deassert the o_lane_success and stop detection.
- If we are not in training mode, the descrambled data will be sent to o_data_out

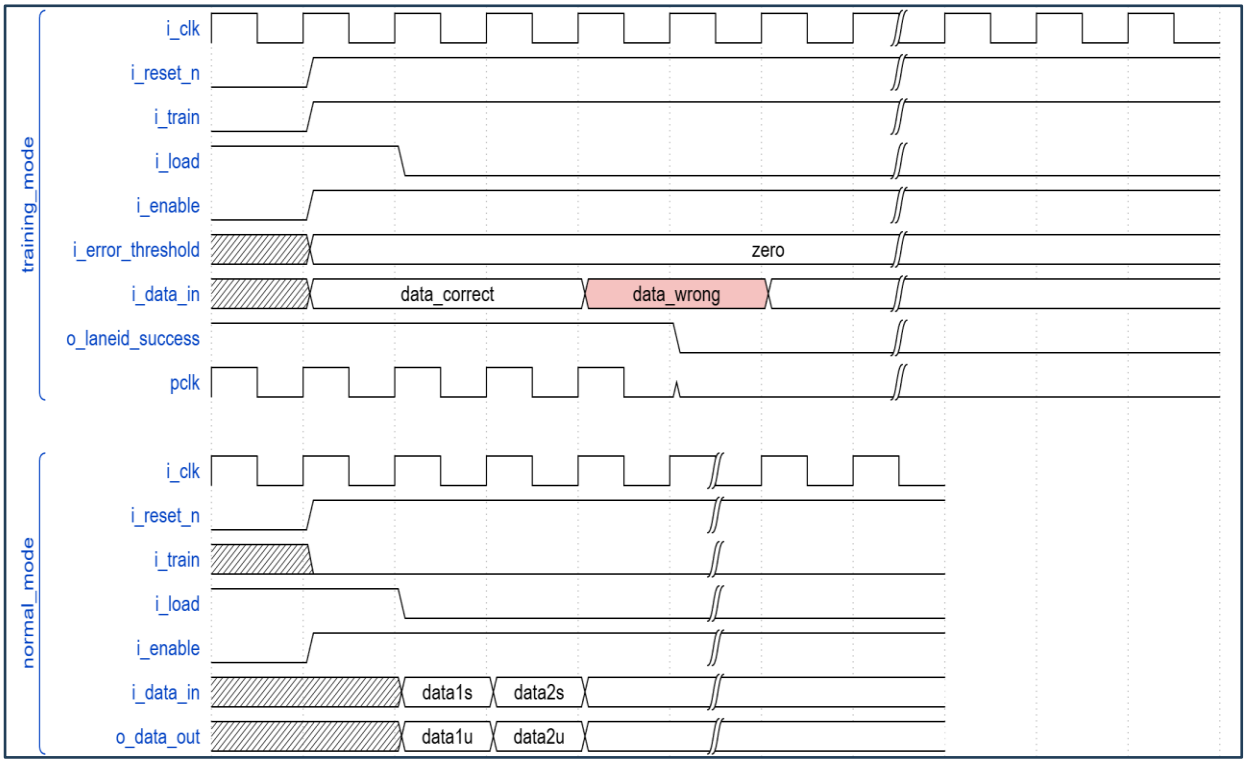
4.3 Block Diagram



Lane_id seed:
The seed of the LFSR is Lane dependent, and based on the Logical Lane number and the seed value for Lane number is modulo 8

Lane	Seed
0	23'h1DBFBC
1	23'h 0607BB
2	23'h1EC760
3	23'h18C0DB
4	23'h010F12
5	23'h19CFC9
6	23'h0277CE
7	23'h1BB807

4.4 Timing Diagram



4.5 Verification Requirements

4.5.1 Verify in training mode that the detection detects if the data has error or not

4.5.2 Verify in training mode that if the error count exceeds the threshold the o_lane_success deasserts

4.5.3 Verify in training mode that the counter stays idle in case of no error

4.5.4 Verify in normal mode that the data gets descrambled correctly